

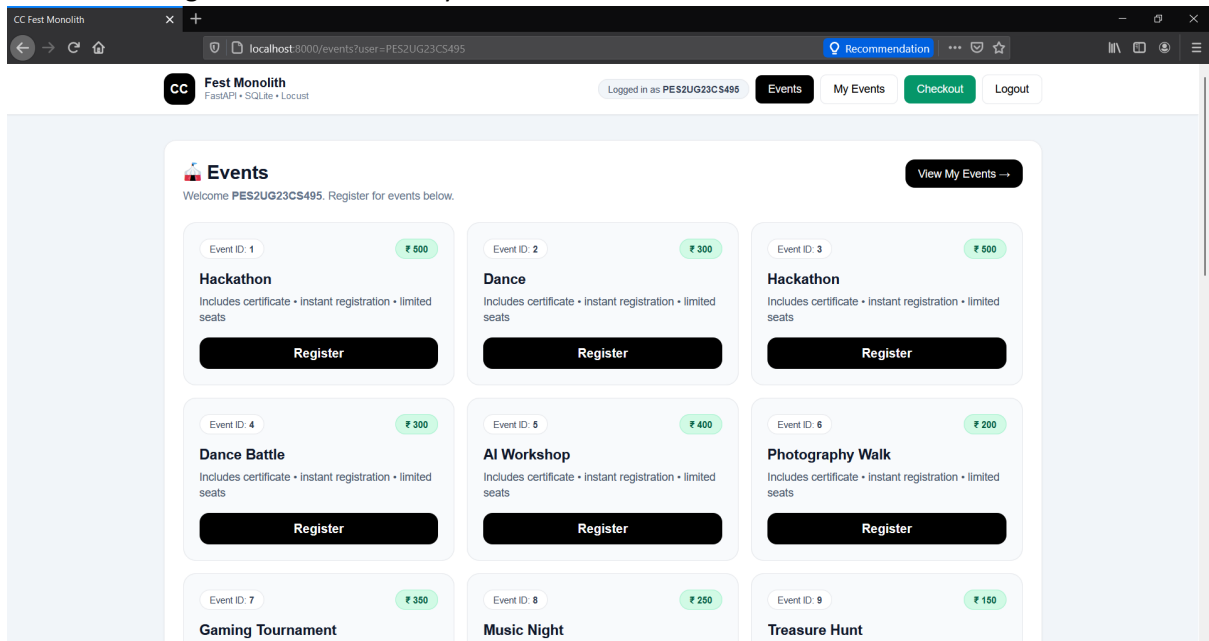
UE23CS351B - CLOUD

COMPUTING

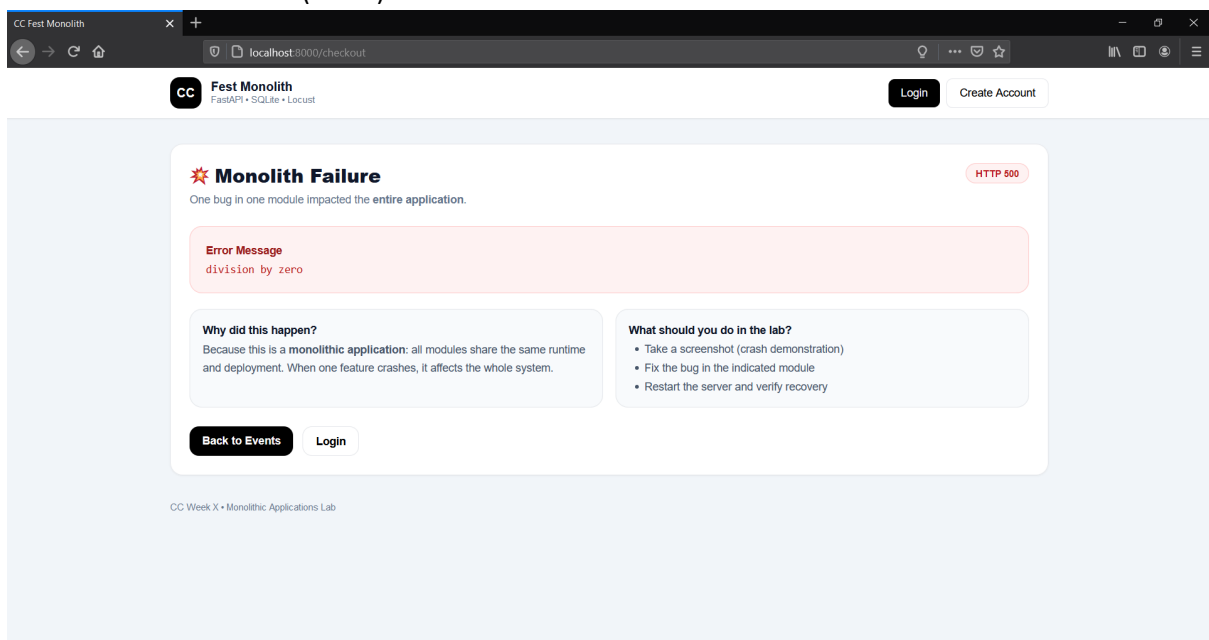
LAB 2

Name: Roshan K C
SRN:
PES2UG23CS495
Date: 29/01/26

SS-1: Events Page Loaded Successfully



SS-2 : Monolithic Failure (Crash)



SS-3 : Checkout Page Fixed

CC Fest Monolith

localhost:8000/checkout

FastAPI • SQLAle • Locust

LoginCreate Account

Checkout

This route is used to demonstrate a monolith crash + optimization.

Total Payable
₹ 6600

✓ After fixing + optimizing checkout logic, re-run Locust and compare results.

CC Week X • Monolithic Applications Lab

What you should observe

- One buggy feature can crash the entire monolith.
- Inefficient loops cause high response times under load.
- Optimization improves performance but architecture still scales as one unit.

Next Lab: Split this monolith into Microservices (Events / Registration / Checkout).

SS-4 : Checkout Load Test (Before Optimization)

CC Fest Monolith

Locust

localhost:8009

67%

STATUS CLEANUP RPS 0.7 Failures 0%

EDIT STOP RESET

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95thile (ms)	99thile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/checkout	19	0	5	2100	2100	112.66	4	2051	2797	0.7	0
Aggregated		19	0	5	2100	2100	112.66	4	2051	2797	0.7	0

```
(C:\Users\Surya H\Documents\all programs\SQL_AleLab-2\FMSS3\SS4\locust> locust -f locust/checkout_locustfile.py
[2020-04-20 09:32:32.152] locust:locust.main: Starting Locust 2.4.3
[2020-04-20 09:32:32.152] locust:locust.main: Starting web interface at http://localhost:8009, press enter to open your default browser.
[2020-04-20 09:32:32.285] locust:locust.runner: Ramping to 1 users at a rate of 1.00 per second
[2020-04-20 09:32:32.285] locust:locust.runner: All users spawned! (Checkpoints: 1) (1 total users)
Feedback (most recent call last):
[2020-04-20 09:32:32.285] locust:locust.runner: www/Lib/site-packages/gevent/_ffi/loop.py", line 276, in python_check_callback
def python_check_callback(callback, watcher_ptr): # pylint:disable=unused-argument

KeyboardInterrupt
[2020-04-20 09:34:04.889] locust:locust.main: Shutting down (exit code 0)
Type      Name      # reqs    # fails    Avg      Min      Max      Med    req/s    failures/s
-----
GET /checkout 19         0         5         2100    2100    112.66    4      2051    2797    0.7      0
Aggregated 19         0         5         2100    2100    112.66    4      2051    2797    0.7      0
```

SS-5 : Checkout Load Test (After Optimization)

CC Fest Monolith

Locust

localhost:8009

67%

STATUS RUNNING Users 1 RPS 0.6 Failures 0%

EDIT STOP RESET

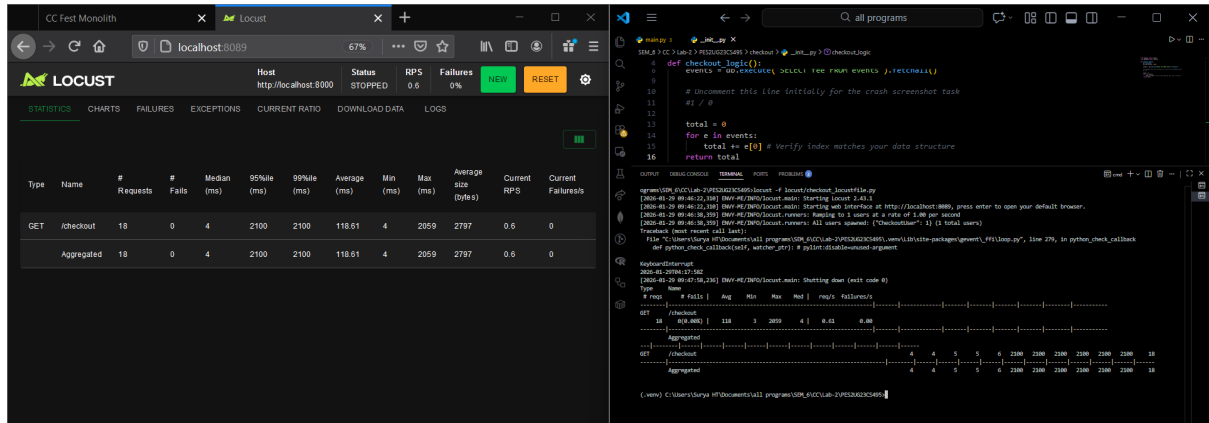
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95thile (ms)	99thile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/checkout	74	0	4	10	2000	33.03	4	2045	2797	0.6	0
Aggregated		74	0	4	10	2000	33.03	4	2045	2797	0.6	0

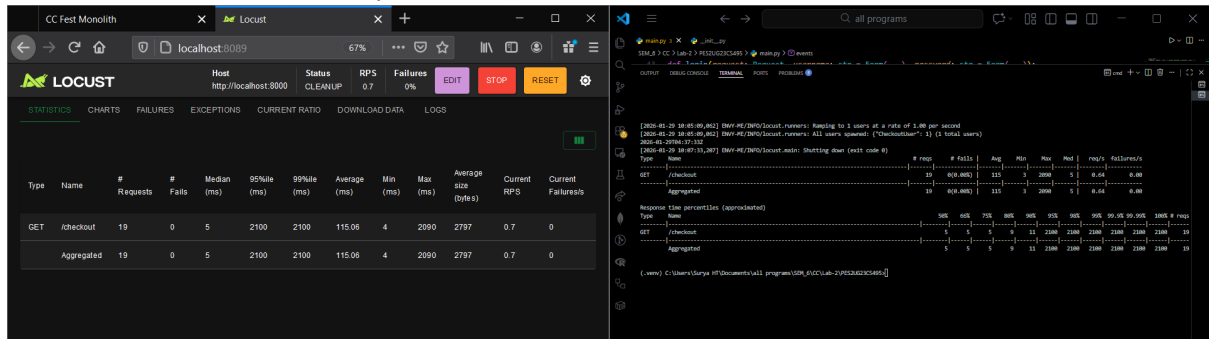
```
(C:\Users\Surya H\Documents\all programs\SQL_AleLab-2\FMSS3\SS4\locust> locust -f locust/checkout_locustfile.py
[2020-04-20 09:38:22.807] locust:locust.main: Starting Locust 2.4.3
[2020-04-20 09:38:22.807] locust:locust.main: Starting web interface at http://localhost:8009, press enter to open your default browser.
[2020-04-20 09:38:36.481] locust:locust.runner: Ramping to 1 users at a rate of 1.00 per second
[2020-04-20 09:38:36.481] locust:locust.runner: All users spawned! (Checkpoints: 1) (1 total users)
Feedback (most recent call last):
[2020-04-20 09:38:36.481] locust:locust.runner: www/Lib/site-packages/gevent/_ffi/loop.py", line 276, in python_check_callback
def python_check_callback(callback, watcher_ptr): # pylint:disable=unused-argument

KeyboardInterrupt
[2020-04-20 09:39:33.191] locust:locust.main: Shutting down (exit code 0)
Type      Name      # reqs    # fails    Avg      Min      Max      Med    req/s    failures/s
-----
GET /checkout 74         0         4         10     2000    33.03    4      2045    2797    0.6      0
Aggregated 74         0         4         10     2000    33.03    4      2045    2797    0.6      0
```

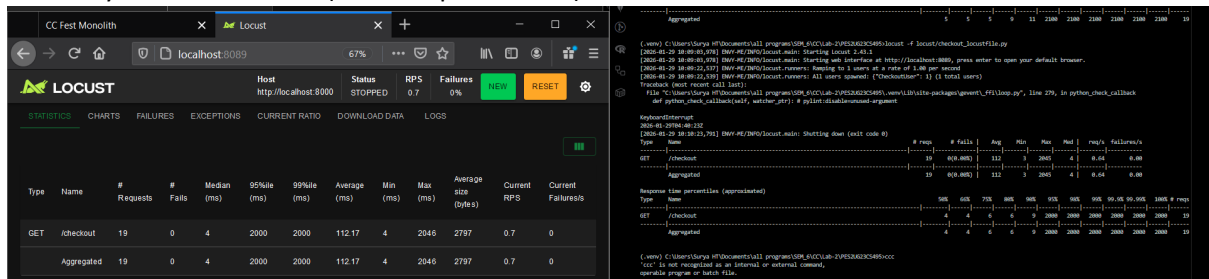
SS-6 : Events Load Test (Before Optimization)



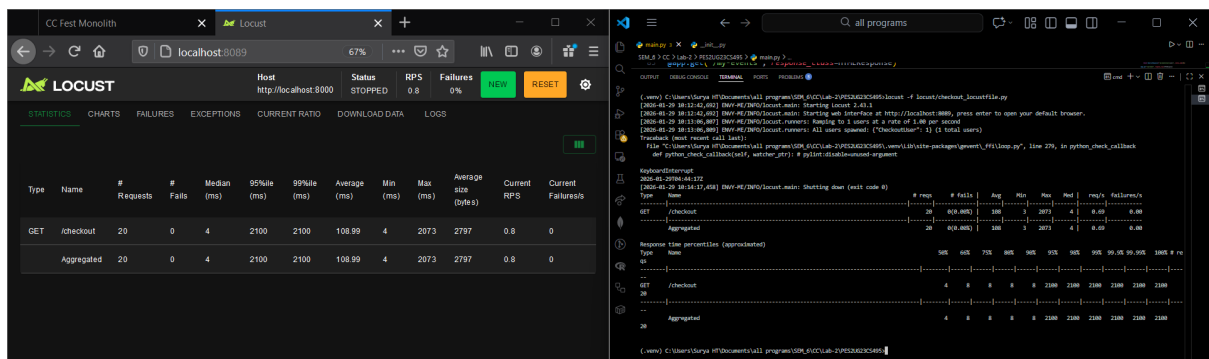
SS-7 : Events Load Test (After Optimization)



SS-8 : My Events Load Test (Before Optimization)



SS-9 : My Events Load Test (After Optimization)



1. What was the bottleneck?

- **/checkout:** An inefficient while loop that incremented and decremented values one unit at a time, causing excessive CPU usage and $O(N)$ time complexity based on the fee amount.
- **/events:** A busy-wait loop running three million iterations with a modulo operation, unnecessarily blocking the server.
- **/my-events:** A blocking loop running 1.5 million iterations with a dummy increment, adding artificial delay to each request.

2. What change did you make?

- **/checkout:** Replaced the iterative loop with a direct arithmetic calculation that sums event prices efficiently.
- **/events:** Removed the busy-wait loop and its related variable entirely.
- **/my-events:** Removed the dummy loop, leaving only the required SQL JOIN operation.

3. Why did the performance improve?

- **/checkout:** Reduced time complexity from being dependent on the fee value to being dependent only on the number of items, enabling instant computation.
- **/events:** Eliminated three million unnecessary CPU operations, significantly reducing response time.
- **/my-events:** Removed 1.5 million redundant iterations, allowing the endpoint to respond immediately and improving throughput (RPS).